



UD3

INTRODUCCIÓN A JAVASCRIPT

3.7 JavaScript asíncrono

MP_0373

Lenguaje de marcas y
Sistemas de gestión de
la información

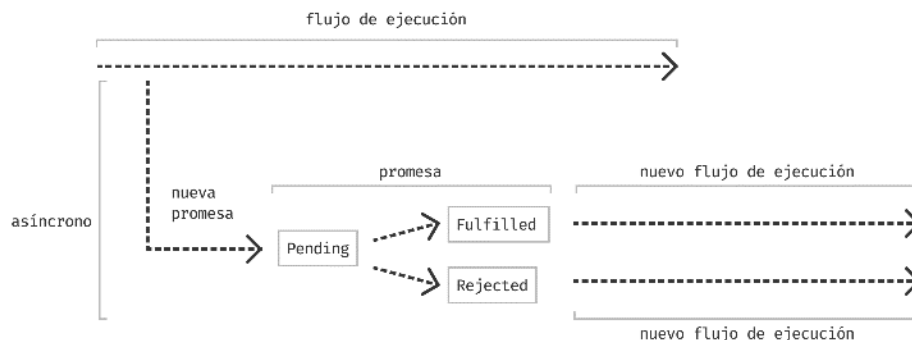
Introducción

El término asíncrono se refiere a que ocurre más de una cosa al mismo tiempo. Pensando en términos de programación: la existencia de varios hilos/flujos de ejecución ocurriendo al mismo tiempo.

JavaScript es un lenguaje de programación pensado para trabajar en asíncrono de una manera sencilla.

Promesas

Una promesa lanza código de **manera asíncrona**, es decir **sale del flujo de ejecución para ejecutarse en otro flujo**. Al igual que el flujo principal, una promesa puede funcionar bien o puede tener algún error en la ejecución.



Una promesa empieza en el estado pending, es decir el proceso todavía no se ha completado y hay que esperar para que se complete. Si dicho proceso se completa correctamente la promesa pasará al estado fulfilled pero si ocurre algún error que hace que el proceso no se haya podido completar, la promesa acabará con un estado rejected.

Por ejemplo, una petición a un servidor puede ser codificada como una promesa, donde al lanzar la petición la promesa se queda en pending, si los datos llegan correctamente la promesa pasará al estado fulfilled pero si hay algún error en el proceso, como que se ha cortado la conexión o que el servidor no responde la promesa acabará como rejected.

```
const promesa = new Promise(function(resolve, rejected) {  
  // proceso  
  
  // si el proceso va bien  
  resolve()  
  
  // si el proceso va mal  
  rejected()  
});
```

En JavaScript una promesa se codifica utilizando la clase Promise. En el constructor de dicha clase se añade como argumento una función que recibe dos parámetros: resolve y rejected. Ambos parámetros son funciones que se deben ejecutar para que la función acabe en fulfilled o en rejected.

La promesa entonces al crearse mediante su constructor empezará a ejecutar en otro flujo de ejecución a la vez que el flujo de ejecución principal (esto se conoce como ejecución asíncrona) el código que haya dentro de la función. Si en ese código va todo bien y se realiza la tarea correctamente, la promesa para comunicarse con el exterior cambia su estado a fulfilled utilizando para ello la función resolve(), en caso contrario, basta con ejecutar rejected() para comunicar que ha habido un error y la promesa pasará al estado rejected.

Then / catch / finally

Las promesas son más útiles si tras ellas se sigue ejecutando otro flujo, que actúe en consecuencia. Para ello, las promesas incorporan los métodos then(), catch() y finally(). Estos métodos ejecutan la función que le llega por argumento:

- ⊕ then() si la promesa ha funcionado bien.
- ⊕ catch() si la promesa no ha funcionado correctamente.
- ⊕ finally() se ejecutará de todos modos, haya ido bien o mal la promesa.

```
const promesa = new Promise(function(resolve, rejected) {  
  resolve("la promesa ha ido bien");  
});  
  
promesa.then(function(result){  
  console.log(result);  
});  
  
// Mostrará por consola:  
// la promesa ha ido bien  
  
// estos métodos devuelven la promesa en sí de nuevo, por lo que pueden  
concatenarse  
promesa.then(function(result){  
  console.log(result);  
}).finally(function(result){  
  console.log("la promesa se ejecutó")  
});  
  
// Mostrará por consola  
// la promesa ha ido bien  
// la promesa se ejecutó
```

Async / await

Utilizar la palabra reservada `async` convierte una función en una función asíncrona: saldrá del flujo de ejecución y se ejecutará en otro flujo.

```
async function funcionAsincrona() {  
    
}  
  
funcionAsincrona(); // Promise
```

De esta manera, la ejecución de la función por sí misma se convierte en una promesa. Tanto es así que devuelve una promesa que dependiendo de las necesidades puede ser tratada o no (es decir, esperar si se ha ejecutado correctamente o no). Si se quiere tratar la función se devuelve

directamente la resolución o error de una promesa y de esta manera se puede enganchar a un `.then()` o `.catch()`:

```
async function funcionAsincrona() {  
  console.log('Función asíncrona');  
  return Promise.resolve("ok");  
}  
  
funcionAsincrona().then(function(result){  
  console.log(result);  
})  
  
// Mostrará por consola:  
// Función asíncrona  
// ok
```

Peticiones con fetch

Uno de los casos de uso de funciones asíncronas por excelencia en una aplicación web es la consulta de datos a un servidor: el código del flujo principal ejecuta una consulta a un servidor, la petición se ejecuta en otro flujo y puede pasar que funcione todo bien y la promesa se resuelva correctamente o que, por ejemplo, el servidor no responda por lo que la promesa falle.

Para realizar peticiones HTTP JavaScript ofrece el método `fetch` que realiza la petición y devuelve simplemente una promesa para tratarla en el código.

```
fetch(recurso).then(function(response) {  
  // La petición se ha realizado  
  // La respuesta está disponible en response  
});
```

Esa respuesta que da `fetch` no es simplemente el contenido, es una instancia de la clase `Response` de JavaScript, la cual provee propiedades y métodos para tratar la respuesta, algunas de ellas son:

Propiedades:

- ⊕ status: el código de estado de la petición HTTP
- ⊕ ok: Booleano cuyo valor es true si el código de estado de la petición es 2XX.

Métodos:

- ⊕ text(): Devuelve una promesa que una vez resuelta devuelve la respuesta en formato texto.
- ⊕ json(): Devuelve una promesa que una vez resuelta devuelve la respuesta en formato json.

// Código típico de petición fetch a servidor que devuelve JSON

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(function(response) {
    return response.json()
  })
  .then(function(posts) {
    // En el parámetro post está un array de posts
    // listo para usar
  });
```

// Utilizando arrow functions

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(response => response.json())
  .then(posts => {
    // En el parámetro post está un array de posts
    // listo para usar
  });
```

El servicio online JSONPlaceholder (<https://jsonplaceholder.typicode.com/>) ofrece una API REST a la que realizar peticiones HTTP con el fin de aprender o probar desarrollos. Es muy similar a como sería un servidor real y devuelve información suficiente para poder probar todo tipo de llamadas.

El método fetch tiene muchas más opciones que las que se han visto en esta documentación, es una pieza fundamental en cualquier proyecto de hoy en día y conviene echar un vistazo a su documentación para ser consciente de lo que es capaz.

MDN Contributors. (2022). *Uso de Fetch* [documentación en línea].
https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Formato de intercambio de datos JSON

JSON es el formato por excelencia en JavaScript para realizar intercambios de datos como en una petición HTTP. Su nombre viene de JavaScript Object Notation y se parece muchísimo a como se escriben y se representan en pantalla objetos en JavaScript. JSON es casi un objeto pasado a String.

```
const persona = {
  nombre: 'Mercer',
  apellido: 'Roussel',
  direccion: 'Cruce Casa de Postas, 33'
};

console.log(typeof persona); // object

const personaJSON =
'{"nombre":"Mercer","apellido":"Roussel","direccion":"Cruce Casa de Postas, 33"}';
console.log(typeof personaJSON); // string
```

De esta manera, se pueden pasar datos estructurados mediante un string que es más fácil de enviar y recibir. Un flujo típico de petición podría ser el siguiente:

- ⊕ El cliente realiza una petición.
- ⊕ El servidor convierte el objeto en memoria a un string que contiene un JSON.
- ⊕ El servidor envía un string.
- ⊕ El cliente recibe un string.
- ⊕ El cliente convierte ese string en un objeto con el que ya puede trabajar.

Para trabajar con JSON, JavaScript provee los siguientes métodos:

- ⊕ `JSON.parse(cadenadetexto)`: Convierte una cadena de texto en un objeto.
- ⊕ `JSON.stringify(objeto)`: Convierte un objeto en una cadena de texto utilizando el formato JSON.